

VaR Risk Engine Development Roadmap

Goal

Build a portfolio risk engine from scratch implementing:

- Historical Simulation VaR
- Variance-Covariance (Parametric) VaR
- Monte Carlo VaR

with minimal coding assistance and maximum learning.

Phase 0: Understand the Risk Problem

Before writing code, be able to answer:

- What is Value at Risk (VaR)?
- What does 95% VaR mean?
- What does 99% VaR mean?
- Why do firms use VaR?
- What are the limitations of VaR?
- What is Expected Shortfall (CVaR)?

Resources

- Philippe Jorion — *Value at Risk*
- John Hull — *Options, Futures, and Other Derivatives*

Target time: 2–3 days.

Phase 1: Design Before Coding

Create a design document before writing code.

Example architecture:

Portfolio

- Positions
- Market Data
- Return Calculator

Risk Engine

- Historical VaR
- Parametric VaR
- Monte Carlo VaR

Backtesting

- Exception Counter
- Kupiec Test

Reporting

- Charts
- Tables
- Export

Questions to answer:

- What are the inputs?
- What are the outputs?
- Which classes should exist?
- How should data flow through the system?

Think like a software engineer first.

Phase 2: Portfolio and Market Data

Build:

- Position
- Portfolio
- MarketDataLoader

Capabilities:

- Load price data from CSV
- Load portfolio weights
- Calculate returns
- Aggregate portfolio returns

Topics to learn:

- Pandas
- Log returns
- Arithmetic returns
- Portfolio construction

Resources

- Pandas Documentation
 - *Python for Data Analysis* — Wes McKinney
-

Phase 3: Historical Simulation VaR

Implement your first risk model.

Process:

1. Calculate portfolio returns.
2. Sort returns.
3. Find the appropriate percentile.
4. Report VaR.

Concepts:

- Percentiles
- Quantiles
- Tail risk

Challenge:

Implement from first principles rather than following a tutorial.

Phase 4: Variance-Covariance VaR

Learn:

- Covariance
- Correlation
- Portfolio variance
- Portfolio volatility

Implement:

1. Covariance matrix calculation
2. Portfolio volatility calculation
3. Parametric VaR estimation

Resources:

- John Hull
- Introductory portfolio theory materials

- Statistical learning references
-

Phase 5: Monte Carlo Engine

Build a correlated simulation engine.

Workflow:

Historical Returns → Covariance Matrix → Correlated Simulations → Portfolio P&L Distribution → VaR Estimate

Topics:

- Random variables
- Multivariate normal distributions
- Cholesky decomposition
- Scenario generation

Resources:

- John Hull
- Paul Glasserman — *Monte Carlo Methods in Financial Engineering*
- NumPy documentation

Focus on understanding why the simulation works, not just implementing it.

Phase 6: Expected Shortfall (CVaR)

Extend the engine to calculate:

- Historical Expected Shortfall
- Parametric Expected Shortfall
- Monte Carlo Expected Shortfall

Understand:

- VaR identifies a threshold.
 - Expected Shortfall measures the average loss beyond that threshold.
-

Phase 7: Backtesting

Validate model performance.

Build a table containing:

- Date
- Predicted VaR
- Actual P&L
- Exception Flag

Calculate:

- Exception Rate
- Kupiec Test
- Basel Traffic-Light Classification

This is where the project begins to resemble a real risk management system.

Phase 8: Stress Testing

Implement historical stress scenarios.

Examples:

- 2008 Financial Crisis
- COVID Crash (2020)
- Black Monday (1987)

Questions:

- How much would the portfolio lose?
 - Which positions contribute most to losses?
 - How does stress loss compare to VaR?
-

Phase 9: Build a Real Application

After the analytics are complete:

Backend:

- Python
- Flask

Frontend:

- HTML
- Bootstrap

Database:

- DuckDB or PostgreSQL

Features:

- Upload portfolio
 - Run VaR calculations
 - Run stress tests
 - Generate reports
 - Visualize distributions and risk metrics
-

Recommended Reading Order

1. Fluent Python — Luciano Ramalho

Purpose:

- Improve software engineering skills.
- Learn Python architecture and design patterns.

2. Options, Futures, and Other Derivatives — John Hull

Purpose:

- Learn portfolio risk concepts.
- Understand risk measurement frameworks.

3. Value at Risk — Philippe Jorion

Purpose:

- Learn professional VaR methodology.
 - Understand how institutions implement and use risk models.
-

Personal Rule

Do not allow AI to write the implementation.

Use:

- Documentation
- Books
- Research papers

- Error messages

Use AI only for:

- Conceptual explanations
- Architecture reviews
- Code reviews after implementation

The goal is not just to finish the project. The goal is to understand every major component well enough that you could explain, modify, or rebuild it yourself.